COURSE ML101 / MODULE ML

ML Fundamentals

Learning goal: explain core concepts, compare related methods, and apply them to realistic machine learning or educational technology scenarios.

Document ID: ML101-01_ML_Fundamentals | Format: instructional Q&A | Revision: 1.0

01. What is machine learning?

Machine learning uses examples to fit a model that maps inputs to predictions or discovers patterns. Unlike a hand-written rule for every case, parameters are learned from data. A spam classifier, for example, learns from labeled messages. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

02. How do supervised and unsupervised learning differ?

Supervised learning trains against known targets, such as house prices or spam labels. Unsupervised learning has no target labels and seeks structure, such as customer clusters or compressed representations. The evaluation method should match the objective. A useful way to understand how do supervised and unsupervised learning differ is to compare the methods not only by definition, but also by the kind of input they expect, the output they produce, and the situations in which each is useful. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

03. What are features, labels, and examples?

An example is one observation. Features are the input measurements available when making a prediction; the label is the desired outcome. In predicting exam success, prior quiz scores may be features and pass or fail may be the label. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

04. Why separate training, validation, and test sets?

Train data fits parameters; validation data supports model and hyperparameter choices; test data estimates final performance on unseen examples. Reusing the test set during tuning makes its estimate optimistic. Split by student or by time when repeated records could leak across sets. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

05. What are overfitting and underfitting?

An underfit model misses important patterns and performs poorly even on training data. An overfit model learns training-specific noise and performs much worse on unseen data. Compare train and validation performance, then adjust complexity, regularization, or data quality. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

06. Explain the bias-variance tradeoff.

High bias means restrictive assumptions cause systematic errors; high variance means predictions change substantially with the training sample. Increasing model complexity can reduce bias while raising variance. Validation results, rather than a fixed rule, guide the choice. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

07. What is data leakage?

Leakage occurs when training uses information unavailable at prediction time, or when test information influences preprocessing and tuning. A future exam grade accidentally included among present-day student features creates misleading performance. Fit preprocessing only on training data within each split. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.